Diagrams

93 terms · name, then the phrase to paste · Workbook Library

Structure & architecture

System context diagram a C4 system context diagram: one box for our system in the center, people as person shapes, external systems as grey boxes around it; every box shows name, [type] and a one-line purpose, every arrow a verb label, and no internals

Container diagram a C4 container diagram for one system inside a labeled boundary: each container as a box with name, [Container: technology] and its job, databases as cylinders, external systems grey outside the boundary; label every arrow with intent and protocol, e.g. reads accounts [JSON/HTTPS]

Component diagram a C4 component diagram zoomed into one container: its boundary as a labeled dashed box, 3 to 7 components inside as boxes with name and [Component], neighbouring containers and systems grey outside, arrows labeled with what is called

Code diagram a C4 code-level view of one component as a UML class diagram: only the 3 to 6 classes and interfaces that matter, public methods only, dependency arrows dashed, implementation arrows with hollow heads, nothing from other components

System landscape diagram a C4 system landscape: all software systems of the organization as boxes inside an enterprise boundary, grouped by business domain, people outside on the left; arrows only for the main relationships, each with a short verb label

Dynamic diagram a C4 dynamic diagram for one scenario: only the containers or components involved, arrows numbered 1, 2, 3 in call order with a short label each, and the scenario name as the title; no more than 8 steps

Deployment diagram a C4 deployment diagram for production: deployment nodes as nested solid boxes labeled with name and [technology], e.g. region > cluster > service; container instances inside with an instance count; arrows labeled with protocol

C4 notation conventions C4 notation conventions: each box shows its name in bold, [element type: technology] beneath, then a one-sentence description; arrows one-way with a verb phrase and [protocol]; a title stating scope and level, and a key for shapes, colors and line styles

Box-and-arrow sketch a quick box-and-arrow sketch: one box per running thing with a plain name, arrows in the direction data flows, a short label on every arrow, left to right, under 8 boxes, and one handwritten note on the part under discussion

Layer diagram a layer diagram: full-width horizontal bands from top (presentation) to bottom (infrastructure), each labeled with its name and two example contents, one arrow on the side saying calls go down only, and cross-cutting concerns as a vertical bar on the right

UML class diagram a UML class diagram: three-compartment boxes (name, key attributes, key operations), associations with multiplicities at both ends, composition with a filled diamond on the owner, inheritance with a hollow triangle on the parent, abstract names in italics, under 8 classes

Entity relationship diagram an entity relationship diagram in crow's foot notation: one box per table with its name in caps, columns listed with PK and FK marked, relationship lines with correct cardinality at both ends, a verb on each line, one subject area of at most 8 tables

Dependency graph a dependency graph laid out top to bottom: each module as a small rounded node, arrows from the depending module to the one it imports, stable shared modules at the bottom, any cycle drawn in red and called out

Network topology diagram a network topology diagram: internet on the left, firewall next, each subnet or VLAN as a labeled zone with its CIDR range, a switch per zone and hosts hanging off it with their last octet; plain lines without arrowheads for links

Threat model data flow diagram a threat model data flow diagram: external entities as rectangles, processes as circles, data stores as two parallel lines, labeled flow arrows, trust boundaries as red dashed lines, and a numbered marker on each flow that crosses a boundary for the threat discussion

Cloud architecture diagram a cloud architecture diagram: nested labeled boundaries for region, VPC and availability zones, managed services as labeled boxes in the right subnet, the request path from users to data drawn first and solid, replication and background flows dashed

Interactions over time

Sequence diagram a sequence diagram of one scenario: participants left to right in the order they are first called, the caller or user first, dashed lifelines, solid arrows for calls labeled with the operation, dashed arrows for replies labeled with the result, time running down

Activation bars a sequence diagram with activation bars: a bar on each lifeline from the incoming call to its reply, self-calls as a looped arrow with a second bar offset to the right, and no bar on a participant that is idle

Combined fragments a sequence diagram with combined fragments: alt frames for branches with a [guard] on each section separated by a dashed line, opt for optional steps, loop with its condition, par for parallel work; at most one level of nesting

Sync, async and reply arrows a sequence diagram that distinguishes message types: solid line with filled arrowhead for a blocking call, solid line with open arrowhead for an async message or event, dashed line for a reply, and a small key explaining the three

Notes, actors and grouping boxes a sequence diagram with a stick-figure actor for the human, a shaded labeled box grouping the backend participants, and notes over the lifelines for rules or timings that the message labels cannot carry

Communication diagram a UML communication diagram: objects as boxes with underlined :Type names, plain lines for links, each message as a small arrow next to its link labeled with a nested sequence number and operation, e.g. 1.2: charge(), no more than 10 messages

Happy path beside failure path a pair of sequence diagrams side by side for the same interaction: left the happy path in green, right the failure path in red showing the timeout, the retry with the same idempotency key and the final outcome, with matching participants and order

State machine diagram a state machine diagram: states as rounded boxes named as adjectives or past participles, a filled dot for the initial state, a bullseye for final states, each transition arrow labeled event [guard] / action, and no transitions that the code does not allow

Hierarchical statechart a hierarchical statechart: a composite state as a large rounded box with its name at top left, child states inside with their own initial dot, orthogonal regions separated by a dashed line, and transitions from the composite border that apply to every child

Timing diagram a timing diagram: one row per participant with its name on the left and its states listed as levels, a stepped line showing the state over a shared time axis in milliseconds at the bottom, arrows between rows for causing messages, and a {≤ n ms} bracket for constraints

Timeline a horizontal timeline: one axis left to right with events as dots placed to scale, each with a timestamp and a short label alternating above and below, the impact window shaded, and the key gap called out

Gantt chart a Gantt chart: tasks as rows listed in start order, weeks as columns, one bar per task, dependency arrows from the end of one bar to the start of the next, milestones as diamonds, and a dashed today line

Lifecycle loop a lifecycle loop: 4 to 6 stages as labeled pills spaced evenly around a ring, curved arrows clockwise from each stage to the next, the arrow back to the first stage highlighted and labeled with what feeds back

Process & flow

Flowchart a flowchart read top to bottom or left to right, with rounded pills for start and end, rectangles for steps written as verb phrases, diamonds for yes or no questions with both exits labeled, and no more than about 12 shapes

Decision tree a decision tree drawn top down, with each question in a box, each branch labeled with its answer, and every leaf a colored outcome pill; keep it to three levels and make sure every path ends in a leaf

Swimlane diagram a swimlane diagram with one horizontal lane per role, lane names in a header strip on the left, steps as boxes in the lane of whoever does them, decisions as diamonds, time flowing left to right, and every lane-crossing arrow treated as a handoff

UML activity diagram a UML activity diagram flowing left to right, starting at a filled dot and ending at a bullseye, with rounded action boxes, a thick fork bar where steps run in parallel and a matching join bar where they all must finish

BPMN process diagram a BPMN 2.0 process inside one pool with a lane per role, a thin-circle start event, rounded tasks named verb plus noun, an exclusive gateway (diamond with X) with labeled outgoing flows, and a thick-circle end event on every path

Data flow diagram a data flow diagram with external entities as squares, numbered processes as circles, data stores as open-ended rectangles with an id, and every arrow labeled with the data it carries as a noun; no control flow, timing or decisions

Customer journey map a customer journey map with 4 to 6 stage columns left to right, rows for what the customer does and their touchpoint, and an emotion curve underneath with a dashed neutral line; mark low points as pain points with a one-line note

Service blueprint a service blueprint with columns for each step left to right and rows for customer actions, frontstage, backstage and support systems, separated by labeled lines of interaction, visibility and internal interaction, with vertical arrows where a row triggers the one below

Value stream map a value stream map with process boxes left to right, a data box under each with process time, inventory triangles between steps, and a timeline ladder along the bottom showing wait time on the high steps and work time on the low steps, totaled at the end

User story map a user story map with user activities as a backbone across the top in journey order, user tasks under each, story cards hanging beneath in priority order, and dashed horizontal release lines labeled MVP and Release 2 that cut across every column

Pipeline diagram a pipeline diagram with stages as chevrons left to right, the jobs of each stage stacked beneath with a pass or fail mark, gates and manual approvals drawn as their own step, and stages after a failure shown dashed as blocked

EventStorming board an EventStorming board read left to right as a timeline: orange past-tense domain events, blue commands before the events they cause, pale yellow aggregates under each command and event pair, lilac policies that react to an event, and pink hot spots for open questions

Ideas & concepts

Mind map a mind map with the central topic in the middle, 4 to 6 main branches radiating out as thick curved limbs in one color each, single words or short phrases on the lines, thinner twigs for details, and no more than about 30 words

Concept map a concept map with each concept as a short noun in a rounded box, arrows labeled with a verb phrase so every link reads as a sentence from one concept to the next, a few cross-links between distant concepts, and no unlabeled arrows

Tree diagram a tree diagram drawn top down with the root at the top, one box per node with its name and role, siblings aligned on one level, parents joined to children by right-angle elbow connectors, and the same box size on each level

Issue tree an issue tree read left to right, with the key question at the root on the left, each level splitting into two to four branches that are mutually exclusive and collectively exhaustive, elbow connectors, and the leaves phrased as questions or hypotheses you can test

Fishbone diagram a fishbone diagram with the problem as the head on the right, a horizontal spine, three to six slanted ribs labeled with cause categories above and below, and short specific causes as horizontal twigs off each rib

Affinity map an affinity map with one observation per sticky note in the person's own words, notes grouped into loose clusters with a dashed boundary and a short theme name on top, three to six clusters, and a count of notes per theme

2×2 matrix a 2×2 matrix with clearly named axes and low and high at each end, a named action for each quadrant in its corner, items placed as labeled dots, and the quadrant you want people to act on lightly shaded

Venn diagram a Venn diagram with two or three translucent overlapping circles of equal size in distinct colors, each set named in its own region outside the overlaps, overlaps labeled only where they mean something, and the center region called out

Spectrum a spectrum drawn as one horizontal line with the two extremes named at each end, items placed along it as labeled dots with leader lines, and a dashed arrow for any item you propose to move

Pyramid a pyramid of three to five horizontal bands, widest at the bottom, each band named inside, with the width meaning quantity and each level resting on the one below, plus one side note explaining what changes from bottom to top

Onion diagram an onion diagram of three to five concentric rings with the core in the center, each ring named at its top, the rings shading darker toward the core, and one arrow and note stating what moving inward means

Cycle diagram a cycle diagram of three to six steps evenly spaced around a circle, read clockwise from the top, each step a short verb in a box, curved arrows between neighbours with the last arrow returning to the first, and the loop's name in the center

Iceberg diagram an iceberg diagram with a wavy waterline, a small tip above it naming what is visible, and a larger mass below divided into two or three labeled layers getting deeper and less visible, with a note on the layer you will act on

Wardley map a Wardley map with the user need as the anchor at the top, components as dots on a value chain going down from visible to invisible, the x axis divided into genesis, custom, product and commodity, dependency lines between components, and dashed red arrows for expected evolution

Craft of explaining

One reading direction one reading direction, left to right for the main flow; each step sits to the right of the one before it, branches and error paths drop downward, and no arrow on the main path points back to the left

Grouping with boundaries and proximity group related elements inside labeled dashed boundaries (team, deployment unit or network zone), keep members of a group close together with wider gaps between groups, and say in the boundary label what the grouping means

Label every arrow label every arrow with a short verb phrase or protocol saying what travels along it (for example "HTTPS/JSON", "reads via SQL", "publishes OrderPlaced"); the arrowhead points in the direction the request or data moves

Legend and key add a small legend in the bottom-right corner that explains every color, line style and shape used (for example blue = our services, dashed box = external, dashed line = async event), with no more than five entries

Color that means something use color for one meaning only and say it in the legend (for example green = healthy, amber = degraded, red = down); everything else stays neutral gray, use at most three colors, and repeat the meaning with a label or icon

Highlight one path, gray the rest draw the whole system in light gray, then highlight only the path being explained in one strong color with thicker lines, so the context is visible but the eye lands on that path first

Align to a grid place every box on a shared grid of rows and columns with equal spacing, use the same width and height for elements of the same kind, and route connectors straight or with right-angle bends only

Numbered steps mark each step with a filled numbered circle (1, 2, 3…) placed on the arrow where it happens, in the order events occur, with a short numbered key under the diagram explaining each step in a few words

Progressive build-up show the diagram as a sequence of frames, each adding exactly one new element or arrow in the highlight color while earlier elements turn gray; every element keeps the same position in every frame

Zoom levels split the explanation into views at different zoom levels: a high-level view with the system as one box, then a view that opens that box to show its parts inside a matching labeled boundary; keep one level of abstraction per view and the same names across views

Callouts and annotations add at most three short callouts, each placed beside the element it explains and joined to it with a thin leader line, written in a contrasting color and a handwritten or italic style so they read as commentary on top of the diagram

Small multiples and before/after draw two panels side by side, labeled before and after, at the same scale with every unchanged element in exactly the same position; draw what changed in a highlight color and keep everything else neutral

Exploded view an exploded view: the layers drawn as flat stacked plates pulled apart along one vertical axis, in their real order, with dashed guide lines joining their corners and a label with a leader line beside each layer

Analogy diagram an analogy diagram: a top row with a familiar real-world scene and a bottom row with the technical system in the same layout, each pair joined by a dashed vertical line, plus a one-line note on where the analogy breaks

Explorable explanation an interactive explanation: one diagram or chart with a single labeled control (slider or toggle) that changes one input, the picture and a numeric readout update live as it moves, and a sensible default value plus a static fallback image

Spaghetti diagram Avoid a spaghetti diagram where every element connects to many others with crossing lines, so no single path can be followed from start to end and no grouping is visible

Unlabeled arrows Avoid arrows without labels, where a reader has to guess whether each line means calls, depends on, sends data to or deploys to, and which way it points

Mixed levels of abstraction Avoid mixed levels of abstraction, with whole systems, code classes, servers and database tables drawn side by side as peers in one view, so nobody can tell what sits inside what

Rainbow colors with no meaning Avoid rainbow colors with no meaning, where every box gets a different color and there is no legend saying what the colors stand for, so readers hunt for a pattern that is not there

Wall of text in boxes Avoid a wall of text in boxes, with sentences or bullet lists crammed into shapes in a tiny font so the structure disappears behind the words

Icon soup Avoid icon soup: product icons and logos with no text labels, so readers have to recognize small symbols to know what each element is or which one is your own service

Arrows that mean different things Avoid arrows that mean different things, where the same line style is used for calls, dependencies, data flows and deployments, and arrow direction is not consistent

Stale diagram that drifted from reality Avoid a stale diagram with no date, owner or source file, that still shows retired components and misses ones added since it was drawn, so readers trust a picture of a system that no longer exists

Decorative 3D, shadows and gradients Avoid decorative 3D, drop shadows, gradients and bevels on boxes and arrows that add no information, suggest depth or layering that is not there and print badly

Diagrams as code

Mermaid Mermaid source in a ```` ```mermaid ```` fenced block (GitHub renders it inline), using the right diagram keyword (flowchart LR, sequenceDiagram, stateDiagram-v2, erDiagram); short node ids, labels in double quotes whenever they contain spaces, parentheses or colons, and no HTML or custom CSS in labels

PlantUML PlantUML between @startuml and @enduml; declare every actor, component and database with an alias before using it, write arrows as a --> b : label, set left to right direction for flows, and use the C4-PlantUML include for architecture views

D2 D2 source with direction: right, boundaries as nested container blocks, shape: cylinder for datastores, every connection written as a -> b: label, and short keys with readable labels; reference nested nodes by their full path like cloud.api

Graphviz DOT Graphviz DOT as a digraph with rankdir=LR, node defaults set once (shape=box, style=rounded), shape=cylinder for datastores, a label on every edge, and subgraph cluster_ blocks with a label for each boundary

Structurizr DSL Structurizr DSL: one workspace with a model of people, software systems and containers (each with a technology string) and a description on every relationship, then systemContext and container views using include * and autolayout lr

Excalidraw an Excalidraw-style sketch: rough hand-drawn boxes and arrows, a handwritten font, a dozen elements at most with short labels, one accent color for the important part; if a file is wanted, valid .excalidraw JSON with rectangle, text and arrow elements bound to each other

ASCII and box-drawing diagrams a plain-text diagram for a code comment using Unicode box-drawing characters (┌ ─ ┐ │ └ ┘ ┬) with ▶ and ▼ for arrowheads, aligned for a monospace font, under 80 columns wide, with short protocol labels beside the lines

draw.io a draw.io diagram as uncompressed mxGraph XML (mxfile, diagram, mxGraphModel, root), each shape an mxCell with vertex="1", a style and mxGeometry, each arrow an mxCell with edge="1" and source and target ids, and labels in the value attribute

Diagram source in the repo keep the diagram source as a text file in the repo next to the docs that use it (for example docs/diagrams/checkout.mmd), render it to SVG in CI, and never commit an exported image without its source

Alt text and a text description give each diagram alt text of one or two sentences stating what it shows and its main relationship, plus a longer text description listing the elements and connections for complex diagrams (in Mermaid, use accTitle and accDescr)

Works in dark mode, print and grayscale make it readable in light mode, dark mode and grayscale print: no hard-coded white background, text and lines with enough contrast on both, and every color difference repeated with a label, line style or shape

Title, caption and date give the diagram a title phrased as the question it answers (for example "How a checkout gets paid") and a one-line caption with its scope, the date or version it reflects and the path to its source file

Generated from code or infrastructure generate the diagram from the source of truth (Terraform, Kubernetes manifests, the import graph or the database schema) in CI, filtered to the elements that answer the question, and flag the pull request when the committed diagram is out of date

Review diagram changes in pull requests treat diagram changes like code: update the diagram source in the same pull request as the change it reflects, include a rendered before-and-after image in the PR description, and give the diagrams a code owner